

测试点 1

对每个 t_i ，暴力与 s_1 进行匹配，如果有任何一个 t_i 是 s_1 的子串，那么答案是 0，否则答案是 1。

时间复杂度： $O(Tm|s_i|)$ 。

测试点 1 ~ 3

还是基于 $n = 1$ 的 10 分做法，可以给 s_1 建一个后缀自动机，然后拿 t_i 一个一个往 sam 匹配，也可以给所有 t_i 建一个 AC 自动机，然后拿 s_1 去跑。总之可以在串长复杂度的时间内判断是否有任何一个 t_i 是 s_1 的子串。

时间复杂度： $O(\sum n + m + |s_i| + |t_i|)$ 。

测试点 4 ~ 5

注意到所有串的长度都是固定的 10，总共可能的串的种类数也就是 $2^{10} = 1024$ ，那么可以构建一个 1024 的图，其中一些节点是不能访问的 (t_i 对应的节点)，一些节点是 s_i 对应的节点，可以给这些节点打上对应的标号，问题本质就是找到一条路径，使得路径上的点的最大标号减去最小标号的差最大。

可以考虑 SCC 缩点 + DAG DP，也可以枚举每个 s_i 对应的点开始记忆化搜索，每个点都记录其往下能到达的最大/最小标号。

理论上可以把数据组数拉到 50000，但实际上这部分数据造得比较随意，随便写点什么暴力一点的做法说不定也可以过。

时间复杂度： $O(T2^{|s_i|} + \sum(n + m + |s_i| + |t_i|))$ 。

测试点 6 ~ 7

注意到所有串都没有 10 这个子串，那么必然是若干个 0 接若干个 1，并且我们注意到把不包含 t_i 的多个 s_i 串拼接起来，这个大串还是不会包含 t_i 的。所以可以给 t_i 建 AC 自动机然后拿 s_i 一个一个去跑，找到最大/最小编号的不包含任何一个 t_i 的 s_i 即可。

实际上更进一步分析，对于这样的串，我们可以把每个串 s 的 0 的个数和 1 的个数统计一下，记作 $(cnt_0(s), cnt_1(s))$ ，那么 s 包含 t 就等价于 $cnt_0(s) \geq cnt_0(t) \wedge cnt_1(s) \geq cnt_1(t)$ ，然后再排个序（可以桶排）扫一遍就行。

时间复杂度： $O(\sum n + m + |s_i| + |t_i|)$ 。

测试点 1 ~ 10

考虑对测试点 4 ~ 5 的做法进行拓展，我们可以把所有的 s_i, t_i 放在一起建 AC 自动机，然后给 t_i 串对应的末尾节点打一个不可访问的标记，同时注意要给 fail 树上的所有子孙节点也打上标记， s_i 同理，这个 dfs 一遍 fail 树就行，然后再 SCC 缩点 + DAG DP 或者枚举起点 + 记忆化搜索来算答案即可。

时间复杂度： $O(\sum n + m + |s_i| + |t_i|)$ 。